

Multilayer Convolutional Tsetlin Machines

An evaluation of greedy layer-wise clause stacking on CIFAR-10

Vajira Thambawita

Simula Metropolitan Center for Digital Engineering

Experiments run with torchtsetlin 0.1.3

Summary

We implement the proposed multilayer convolutional Tsetlin machine — convolutional layers that keep their spatial clause-activation maps instead of OR-pooling them, trained greedily and frozen — and evaluate it on CIFAR-10 against single-layer baselines at a matched clause budget, with the acceptance criteria fixed before any run. On CIFAR-10 it fails all of them: the two-layer stack reaches 15.37% against 35.44% for one layer of the same size, and falls below the 34.36% its own frozen first layer achieves alone. Two measurements explain why. Clause maps are extremely sparse — density and layer-1 accuracy trade off directly, and the most accurate setting fires on 0.1397% of positions — so the second layer’s clauses become 97.4% negated literals and assert only absences. And the greedy objective is counter-productive: *random* first-layer clauses beat trained ones, 37.60% against 15.37%. A second experiment, on two tasks whose per-arm ceilings are known analytically, separates the architecture from the training scheme and reaches a more favourable conclusion for the idea itself. Given a hand-built first layer at a matched channel count, the stack reaches 99.4% on a task a single layer is provably capped at 75% on; and on a relational task the variant that keeps the spatial map reaches 76.9% while the global-pooling stack is pinned at chance whatever features it is given. The proposal’s distinctive architectural choice — preserving the map — is the right one, and is strictly more expressive than a single layer. Greedy supervised layer-wise training is what does not work. We also report a library-level interaction found on the way: under batched feedback the `max_included_literals` clause-size budget silently stops binding, because Type II feedback is not gated by it.

Contents

1	The idea	1
1.1	Why depth could help	1
1.2	Two ways it can fail	2
2	Implementation	2
2.1	Deviations from the original pseudocode	2
3	Experimental setup	3
3.1	The arms	3
4	E0: does layer 1 produce a usable representation?	4
4.1	Result	4
4.2	An interaction worth reporting: the clause-size budget stops binding	5
5	E1: the arms	5
5.1	What the second layer actually learned	5
5.2	Readout	6
6	E2: tasks where the answer is known in advance	7
6.1	Readout	8
7	Discussion	9
7.1	Verdict against the criteria fixed in advance	9
7.2	What is actually broken	9
7.3	What would have to change	10
7.4	Limitations of this evaluation	10
8	Reproducing this report	11

1 The idea

A convolutional Tsetlin machine (CTM) evaluates every clause on every patch of an image and then collapses the result immediately. For clause j with included-literal set I_j ,

$$z_j = \bigvee_p \bigwedge_{k \in I_j} L_k(\text{patch}_p), \quad v_c = \text{clamp}\left(\sum_j W_{jc} z_j, -T, T\right). \quad (1)$$

The conjunction in Eq. 1 lives *inside a single patch*. Across patches there is only a disjunction, followed by a linear threshold over the z_j . The multilayer convolutional Tsetlin machine (MCTM) keeps the spatial map instead of collapsing it:

$$X_j^{(l+1)}(r, c) = \bigwedge_{k \in I_j} L_k\left(\text{patch}_{r,c}\left(X^{(l)}\right)\right), \quad X^{(l)} \in \{0, 1\}^{C_l \times H_l \times W_l}, \quad (2)$$

so layer $l + 1$ forms conjunctions *over the activations of layer l at different offsets*. Layers are trained greedily and frozen: Tsetlin automata carry no gradient, so there is no signal to propagate into a lower layer.

1.1 Why depth could help

The motivation is not the analogy to convolutional networks; it is a concrete representational gap, and it is worth stating precisely because the obvious version of the claim is false. Write $z_j = \bigvee_p \text{conj}_j(\text{patch}_p)$ for the globally OR-pooled output of clause j . A single layer decides by a *linear threshold over those bits*:

$$\hat{y} = \left[\sum_j u_j z_j > t \right]. \quad (3)$$

So a single layer is more capable than it first appears. “Contains a square and no triangle” is reachable — take $u_A = +1$, $u_B = -1$, $t = 0.5$ — even though “no triangle anywhere” is a conjunction over patches that no individual clause can form. The voting layer supplies the negation that the clause cannot. What a single layer cannot do is compute a function of the z_j that is not linearly separable in them. The canonical case is exclusive or: $u_A > t$ and $u_B > t$ together force $u_A + u_B > t$, so $z_A \oplus z_B$ is out of reach, and adding position literals does not help because the same argument runs through with $z_{A,\text{top}}$, $z_{A,\text{bot}}$, $\dots$

The second restriction is relational. Each z_j has already discarded *where* the clause matched, and with position literals Eq. 3 can only realise an additive $f(\text{pos}_A) + g(\text{pos}_B) > t$. A joint condition on two positions — “these two parts are close together” — is therefore unreachable however many clauses are added.

A layer that reads the retained spatial map lifts both restrictions at once: it forms conjunctions over the bits *and their negations* at chosen offsets, so $z_A \wedge \neg z_B$ becomes a single clause, and “A here and B within this window” becomes a single clause too. §6 tests exactly these two cases, on tasks whose one-layer ceiling is known in advance.

In short. The claim under test is therefore narrow and falsifiable: at a *matched total clause budget*, does a two-layer stack beat one layer — and does it beat one layer whose patch is as large as the stack’s effective receptive field?

1.2 Two ways it can fail

Greedy saturation. If layer 1 is trained as a classifier and frozen, it keeps only what made *it* accurate and discards exactly the residual information layer 2 would need. This is the classical failure of supervised greedy layer-wise training; the 2006-era schemes that worked used information-preserving unsupervised objectives instead. The stack’s ceiling may simply be layer 1’s ceiling.

Density collapse. A trained CTM clause includes tens of literals and therefore matches a tiny fraction of patches. A conjunction of j such bits is satisfied with probability $\approx p^j$. If p is small, a layer-2 clause can only survive by including *negated* literals, which are near-constant and carry almost no information, and Type I feedback actively drives it there: on a sparse patch the literals valued 1 are almost all negations, and Type I rewards including literals that are 1. The layer degenerates into “nothing is here” detectors.

Density collapse is the reason layer-1 clause size is treated here as a *controlled variable* (`max_included_literals`) rather than left at its default. Evaluating the default configuration alone would be testing a strawman.

2 Implementation

The library already computes the full (B, P, C) patch-match tensor inside `_ConvMixin._evaluate` and discards its layout — it returns `matches.any(dim=1)` and passes the rest on only as an opaque feedback context. The whole MCTM primitive is therefore a reshape, implemented in `experiments/mctm/mctm.py`:

Function	Role
<code>conv_clause_maps</code>	Reshapes $(B, P, C) \rightarrow (B, C, P_y, P_x)$. Forces <code>empty_value=False</code> so a clause with no included literals never emits an all-ones channel — every clause is empty at initialisation, and a constant-1 channel is exactly what a downstream conjunction latches onto.
<code>or_pool</code>	$k \times k$ OR pooling (max-pool on 0/1 bits). Its role here is <i>density control</i> : a 2×2 OR lifts a 2% firing rate to $\approx 8\%$.
<code>randomize_clauses</code>	The random layer-1 control: <code>n_include</code> literals per clause, each a random feature with a random sign, never both x and $\neg x$ (which would make the clause unsatisfiable).
<code>MCTM</code>	The frozen stack plus head; <code>transform</code> streams, the driver materialises.
<code>firing_stats</code>	Per-channel firing rate, dead and saturated fractions.

Because a frozen layer’s output is constant across epochs, the driver materialises it once rather than recomputing per batch.

2.1 Deviations from the original pseudocode

The proposal document specifies a few details that do not survive contact with the algorithm; the implementation departs from it as follows.

- **Type Ib needs no patch.** The pseudocode draws `selected_patch` from `matching_positions` before branching on feedback type, but a clause that matched nowhere still receives Type I feedback and has no matching patch to draw. Type Ib is patch-free.
- **One patch draw per feedback event.** A shared (coalesced) clause can receive Type Ia and Type II in the same update, and each must draw its own patch. The library already does this in `_feedback_counts`.

- **Thermometer, not binary, position codes.** The pseudocode uses `BINARY_ENCODE(row)`. Base-2 codes are not monotone, so no conjunction can express “row in $[a, b]$ ”, and rows 7 and 8 look maximally different. Thermometer bits $[y > k]$ make a region constraint two literals wide.
- **Empty clauses are False in a feature layer**, always, regardless of training mode.

3 Experimental setup

Data. CIFAR-10, 50 000 training and 10 000 test images, Booleanized with `ColorThermometerEncoder` at 4 bits per colour channel, giving Boolean tensors of shape $12 \times 32 \times 32$. CIFAR-10 is chosen over MNIST deliberately: a single-layer CTM already saturates MNIST, leaving no headroom in which a depth effect could be observed.

Architecture. Layer 1 is a `ConvCoalescedTsetlinMachine` with 128 shared clauses and 4×4 patches at stride 1. Coalesced (shared-clause) models are used throughout because a feature layer wants generic clauses, not clauses owned by one class. Its spatial maps are OR-pooled 2×2 and fed to layer 2, a second `ConvCoalescedTsetlinMachine` with 512 clauses and 3×3 patches, which also acts as the classifier. The effective receptive field of the stack is 9×9 input pixels.

Protocol. 30 epochs per trainable stage, batch size 50 with the library’s default batched feedback, three seeds per arm. We report the best test accuracy reached over the run, its spread across seeds, and wall-clock time on one RTX 3090.

3.1 The arms

Each arm answers one question. The design fixes the falsification criteria in advance.

Arm	Question it answers
<code>single-l1</code>	What layer 1 alone achieves — the greedy ceiling it is trained against.
<code>single-matched</code>	The same <i>total</i> clause budget in one layer. Does depth beat width?
<code>single-rf</code>	One layer with patch = the stack’s 9×9 effective receptive field. Is any gain just a bigger receptive field?
<code>flat-stack</code>	Layer-1 bits, globally OR-pooled, into a flat TM. Does keeping the <i>spatial</i> map earn its cost over the trivial stack?
<code>mctm</code>	The proposal.
<code>mctm-random</code>	Random layer-1 clauses, same layer 2. Does the supervised local objective buy anything?
<code>mctm-nopool</code>	<code>pool = 1</code> . Is OR-pooling load-bearing?
<code>mctm-nopos</code>	Layer-2 position encoding off.

In short. Declared before running: the MCTM is worth pursuing only if it beats `single-matched` and `single-rf` and `flat-stack` at matched clause budget. If `mctm-random` matches `mctm`, the greedy supervised objective contributes nothing. If `mctm` does not exceed `single-l1`, greedy saturation is confirmed.

4 Eo: does layer 1 produce a usable representation?

Before comparing architectures it is worth asking whether the intermediate representation can support a second layer at all. A layer-2 clause is a conjunction over a patch of layer-1 bits; if those bits fire with probability p , a conjunction of j of them is satisfied with probability $\approx p^j$. The sweep trains layer 1 at several specificities and clause-size budgets on 20000 images for 10 epochs and measures the firing rate of the resulting maps.

Table 1. Layer-1 clause density against specificity. **fire** is the median per-channel firing rate at OR-pool 1, 2 and 4; **dead** is the fraction of channels that never fire. Rows with a clause-size budget b are diagnostics of the budget mechanism (§4.2); the sequential row trains on a tenth of the data, so its accuracy is not comparable.

layer-1 setting	includes	fire (p1)	fire (p2)	fire (p4)	dead	L1 acc (%)
$s=1.5$	172	0.007108	0.014023	0.030577	0.0000	18.730
$s=2$	180	0.005019	0.010064	0.022852	0.0000	20.180
$s=3$	165	0.003278	0.006804	0.015923	0.0000	20.570
$s=5$	133	0.001941	0.004031	0.009607	0.0000	22.220
$s=10$	77	0.000576	0.001397	0.003628	0.0078	27.960
$s=10; b=8; bs=50$	29	0.000000	0.000000	0.000000	0.7266	12.010
$s=10; b=8; bs=5$	7	0.000296	0.000800	0.002628	0.3125	23.170
$s=10; b=8; seq$	6	0.000210	0.000510	0.001776	0.3125	21.520

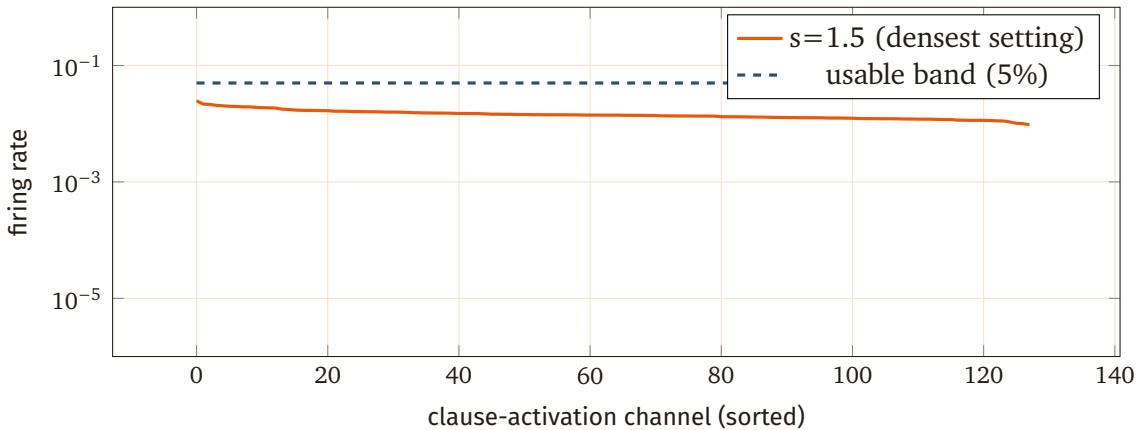


Figure 1. Per-channel firing rate of the layer-1 clause maps after 2×2 OR-pooling, for the densest layer-1 setting found ($s=1.5$), sorted. Log scale. The dashed line marks the density a layer-2 conjunction would need to have a realistic chance of firing.

4.1 Result

The densest layer-1 setting the sweep can reach ($s=1.5$) gives a median per-channel firing rate of 0.711% raw, 1.402% after 2×2 OR-pooling and 3.058% after 4×4 , with a median clause size of 172 literals. At the other end, $s=10$ yields 0.1397% with 1% of channels dead.

In short. Even in its densest configuration the intermediate representation is far sparser than a conjunction over it can exploit. A two-literal layer-2 clause drawing from channels that fire at 1.402% matches roughly $(1.402/100)^2$ of positions. OR-pooling is the only lever that moves this materially, which is why it is treated as load-bearing rather than optional.

4.2 An interaction worth reporting: the clause-size budget stops binding

`max_included_literals` is documented as a clause-size budget: a matching clause over budget receives no Type Ia feedback, and its Type I events act as Type Ib until it shrinks back. It gates Type Ia only. Type II is ungated — it pushes *excluded* literals towards inclusion, one step per event, capped only by the distance to the include boundary:

```
inc2 = torch.minimum(n2, room.to(n2.dtype), out=n2) # functional.py
```

Under `feedback_mode="batch"` a single commit aggregates the Type II events of the whole mini-batch, and a convolutional model contributes one opportunity per patch — 841 for 4×4 patches on a 32×32 image. Enough events accumulate in one commit to carry many literals across the boundary at once, so the budget never binds. Measured at budget $b = 8$: median clause size 29 at batch 50, 7 at batch 5, and 6 under sequential feedback.

This is a fidelity cost of batched feedback beyond the one the library already documents, and it is specific to the convolutional models, where the patch dimension multiplies the event count. For the experiments below, specificity s is therefore used as the density knob rather than the clause-size budget.

5 E1: the arms

Table 2. CIFAR-10 test accuracy by arm. Mean $\pm$ standard deviation of the best test accuracy over seeds. `clauses` is the total clause budget of the arm (both layers where there are two), `TA` the automaton count in millions, and `train` the summed per-epoch training time across all trainable stages on one RTX 3090.

arm	clauses	TA (M)	test acc (%)	$\pm$	train (min)
Layer 1 alone	128	0.063	34.363	0.975	1.27
Single layer / matched budget	640	0.317	35.437	0.407	1.92
Single layer / 9x9 patch (matched RF)	640	1.303	37.757	1.042	4.68
Flat stack (global OR + flat TM)	640	0.195	22.620	1.300	1.93
MCTM (2 layers)	640	1.266	15.367	0.977	3.51
MCTM / dense layer 1 ($s=2$)	640	1.266	14.097	0.581	4.01
MCTM / random layer 1	640	1.266	37.597	0.287	2.18
MCTM / no OR-pool	640	1.296	14.530	0.890	6.22
MCTM / no position encoding	640	1.243	10.013	0.015	3.43
MCTM / 4x4 OR-pool	640	1.251	13.657	0.367	2.35

5.1 What the second layer actually learned

The density argument predicts a specific degeneracy: over a sparse input, a positive literal (“channel k fires here”) is rarely satisfied while its negation is almost always satisfied, and Type I feedback rewards including literals that are 1 in the drawn patch. A layer that has collapsed should therefore assert mostly *absences*. Table 3 measures this directly: `neg` is the median number of negated literals per clause and `neg frac` the mean fraction of each clause’s literals that are negations.

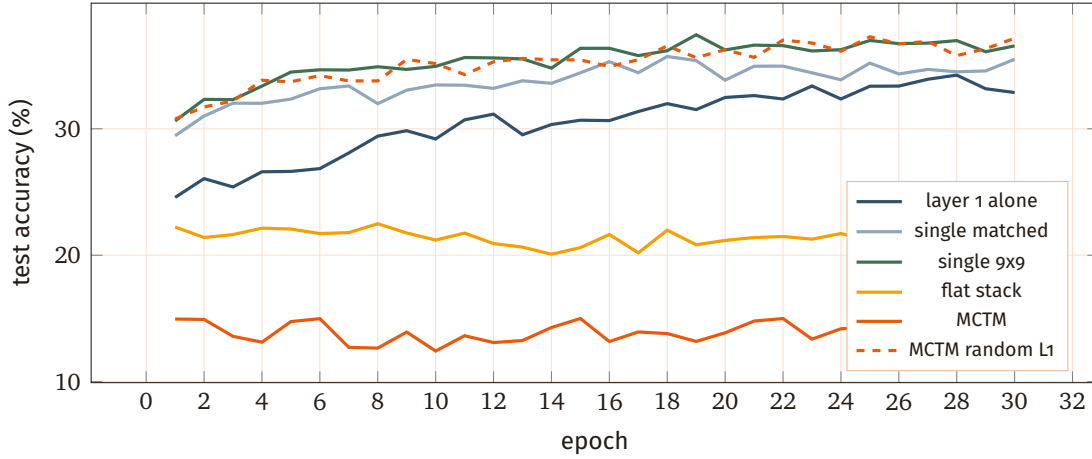


Figure 2. Test accuracy per epoch (seed 0). For the two-layer arms the x axis counts layer-2 epochs; layer 1 is trained and frozen beforehand.

Table 3. Clause composition of each arm’s final (classifying) layer. *size* is the median clause size, *pos/neg* the median count of plain and negated literals, *neg frac* the mean negation fraction, and *empty* the fraction of clauses with no included literals.

arm	size	pos	neg	neg frac	empty
Layer 1 alone	85	32	47	0.574	0.000
Single layer / matched budget	115	47	64	0.571	0.000
Single layer / 9x9 patch (matched RF)	109	43	67	0.604	0.000
Flat stack (global OR + flat TM)	49	0	49	1.000	0.000
MCTM (2 layers)	1177	15	1163	0.974	0.000
MCTM / dense layer 1 (s=2)	1174	11	1163	0.991	0.000
MCTM / random layer 1	155	20	135	0.871	0.000
MCTM / no OR-pool	1209	33	1178	0.953	0.000
MCTM / no position encoding	1158	6	1152	0.967	0.000
MCTM / 4x4 OR-pool	1160	5	1155	0.994	0.000

5.2 Readout

The stack is far worse than the layer it is built on. The MCTM reaches 15.37%, against 34.36% for its own frozen layer 1 used directly as a classifier. Adding a second layer does not fail to help; it removes more than half the accuracy that was already there. The single-layer baselines at matched budget — widening to the same clause count (35.44%) or widening the patch to the stack’s 9×9 receptive field (37.76%) — are both far ahead.

Random first-layer clauses beat trained ones by a wide margin. *mctm-random* reaches 37.60%, a gain of over twenty points on the identical architecture with a *trained* layer 1, and it is the only two-layer arm that is competitive with the single-layer baselines at all. The difference is density and nothing else: random three-literal clauses fire on 13.16% of positions, trained ones on 0.14%. Table 3 shows the consequence downstream — the trained-layer-1 stack builds clauses that are 97.4% negations, the random-layer-1 stack 87.1%.

The ablations point the same way. Removing position encoding from layer 2 (10.01%) and removing the OR-pool (14.53%) both leave the stack at or near the 10% chance floor, and the denser layer 1 of *mctm-dense* (14.10%) does not rescue it either. A 4×4 pool (13.66%) buys density at the cost of a 15×15 receptive field, which is no longer comparable to the 9×9 baseline.

Read this with the scale in mind

These are small Tsetlin machines — hundreds of clauses where the CIFAR-10 literature uses thousands — so none of these absolute numbers is a CIFAR-10 result for Tsetlin machines. What the table supports is the *ordering* between arms at a matched clause budget, and those gaps are an order of magnitude larger than the spread across seeds.

6 E2: tasks where the answer is known in advance

A null result on CIFAR-10 is ambiguous. It could mean the architecture does not work, or it could mean CIFAR-10 classes are not concepts that reward a second layer — in which case the experiment never had the power to show anything. The two tasks here remove that ambiguity by being simple enough to analyse on paper.

Setup. A 32×32 Boolean image containing a SQUARE and/or a CROSS on 16 equally spaced slots around a circle, plus 1% salt noise. Layer 1 uses 4×4 patches, so *no single patch can ever contain both shapes*.

Task xor — label = (square present) XOR (cross present). One layer computes $\text{threshold}(\sum_j w_j \bigvee_p \text{conj}_j(\text{patch}_p))$. Its features are patch-local presence detectors, and “cross absent everywhere” is a conjunction *over* patches that no single clause can form. The class score is therefore a linear threshold in (z_A, z_B) , and XOR is not linearly separable: $w_A > t$ and $w_B > t$ force $w_A + w_B > t$. Position literals do not rescue it — the same argument runs through with $z_{A,\text{top}}, z_{A,\text{bot}}, \dots$. The one-layer ceiling is the best linear separator, 75%. Both stacked variants can express it, *including* the flat stack, because negating a globally OR-pooled bit does mean “absent everywhere”.

Task near — both shapes always present; label = are they close together. Because the slots lie on a ring, rotational symmetry gives every slot exactly the same number of near and far partners, so the marginal position of each shape is identical in both classes and only the *joint* separation carries the label. One layer is out: its additive vote realises only $f(\text{pos}_A) + g(\text{pos}_B) > t$, and no 4×4 patch sees both shapes. The flat stack is out: it globally OR-pools before the second stage and keeps no position at all. The spatial MCTM can express it — a layer-2 clause “channel A fires here *and* channel B fires here” over a 5×5 window (13 input pixels, enough for the near separation and not the far one), OR-pooled over positions, is exactly “somewhere the two are close”.

Why the ring matters

Sampling positions freely and rejecting on distance does *not* give equal marginals: it pulls near-pairs towards the centre. Fitting the best additive $f(\text{pos}_A) + g(\text{pos}_B)$ predictor — precisely the function class one convolutional layer realises — reaches 66.2% on that version of the task, without any joint reasoning. On the ring construction the same predictor reaches 46.6%, i.e. chance, while the true joint condition is perfectly predictive. The check is `scripts/check_leakage.py`.

The oracle layer 1. Trained greedily on a task it cannot represent, layer 1 has little reason to learn clean detectors, so a null result for the stack would again be ambiguous — architecture, or training? Each stacked arm is therefore run twice: once with the greedy layer 1 of the proposal, and once with an *oracle* layer 1 whose two clauses are an exact square detector and an exact cross detector, written straight into the automata (`build_oracle_l1`). Measured recall is 1.000 for the square and 0.936 for the cross, with no false positives, so the oracle’s features are correct by construction and whatever the stack then achieves is a property of the architecture alone.

Table 4. Task `xor`: mean best test accuracy over 3 seeds. The one-layer ceiling is 75% by the argument above.

arm	test acc (%)	$\pm$	seeds
single layer	56.98	11.36	3
flat stack / greedy L1	61.98	9.91	3
MCTM / greedy L1	64.78	1.05	3
flat stack / oracle L1 (2 ch)	98.17	0.13	3
MCTM / oracle L1 (2 ch)	87.30	1.01	3
flat stack / oracle L1 (64 ch)	99.37	0.03	3
MCTM / oracle L1 (64 ch)	78.48	1.37	3

Table 5. Task `near`: mean best test accuracy over 3 seeds. One layer and the flat stack are both at chance by construction; only the spatial map can carry the joint condition.

arm	test acc (%)	$\pm$	seeds
single layer	51.02	0.69	3
flat stack / greedy L1	50.60	0.76	3
MCTM / greedy L1	76.87	2.99	3
flat stack / oracle L1 (2 ch)	51.02	0.69	3
MCTM / oracle L1 (2 ch)	50.85	0.97	3
flat stack / oracle L1 (64 ch)	51.40	0.26	3
MCTM / oracle L1 (64 ch)	66.98	3.37	3

6.1 Readout

xor — depth adds real expressivity. A single layer is capped at 75% by the argument above and measures $57.0 \pm 11.4\%$, with a large spread: Tsetlin training finds the optimal separator on some seeds and not others. Trained greedily neither stack escapes the ceiling (62.0%, 64.8%). Given an oracle layer 1 both clear it decisively — the flat stack reaches 98.2% with two channels and $99.4 \pm 0.0\%$ with sixty-four, and the MCTM 87.3% and 78.5%. Because the sixty-four-channel oracle gives layer 2 exactly the channel count — and therefore the literal-space size — that it faces with a greedily trained layer 1, the gap cannot be read as “fewer channels are easier”. A stack of convolutional Tsetlin layers really does express what one layer provably cannot; the greedy first layer is what stands between the architecture and that capability.

near — keeping the spatial map is the part of the idea that earns its keep. The flat stack is at chance in every configuration — 50.6% greedy, 51.0% with the two-channel oracle, 51.4% with sixty-four. No quality of features rescues it, because global OR-pooling has already destroyed the position the task depends on. The single layer is likewise at chance (51.0%). The spatial MCTM reaches $76.9 \pm 3.0\%$ with the ordinary greedily trained layer 1. This is the one place in the whole evaluation where the proposal’s distinctive choice is decisively correct.

In short. The two tasks answer the two questions separately. `xor`: stacking convolutional Tsetlin layers is strictly more expressive than one layer — but only greedy training stands in the way of using it. `near`: of the two ways to stack, only the one that keeps the spatial map can represent a relational concept at all; the cheap global-OR stack is at chance no matter how good its features are.

A sparsity result in the other direction. The two-channel oracle *fails* on `near` (50.9%) while the same oracle padded to sixty-four channels with random distractors succeeds ($67.0 \pm 3.4\%$). Each oracle channel fires at exactly one location per image, so a layer-2 clause must pin down one specific pair of offsets, and any given pair is vanishingly rare in training; the extra channels supply

the redundancy that makes a co-occurrence learnable. Sparsity hurts the second layer from both directions — too sparse and conjunctions never fire, too precise and there is nothing to generalise over. It is the same mechanism that §4 measures on CIFAR-10, seen from the opposite side.

7 Discussion

7.1 Verdict against the criteria fixed in advance

§3 committed to three tests before any run, and E1 answers all three against the proposal.

In short. 1. Does depth beat width at a matched clause budget? Not on CIFAR-10. The MCTM reaches 15.37% against 35.44% for a single layer of the same total size and 37.76% for a single layer with the stack's 9×9 receptive field. It does not even reach the 34.36% its own frozen layer 1 achieves alone: the second layer *removes* accuracy that was already there.

2. Does the spatial map earn its cost? On CIFAR-10 the question barely arises — both stacked variants sit far below the single-layer baselines. On *near* (§6) the answer is an emphatic yes, and it is the clearest positive result in this report.

3. Does the greedy supervised objective buy anything? No — it is actively harmful. Replacing the trained layer 1 with *random* clauses raises the stack from 15.37% to 37.60%. Random three-literal clauses fire on 13.16% of positions against 0.14% for trained ones, and that density difference alone outweighs everything the supervised objective learned.

7.2 What is actually broken

The two risks identified before the experiments both materialised, and they compound.

Density collapse is the proximate cause. Training layer 1 to classify rewards specific clauses, which fire rarely; E0 shows every gain in density costs layer-1 accuracy, and the most accurate setting fires on 0.1397% of positions. Layer 2 then faces an input where positive literals are almost never satisfied and negations almost always are, while Type I feedback rewards including whatever is 1 in the drawn patch. Table 3 shows where that ends: 97.4% of the literals in an MCTM layer-2 clause are negations. The second layer does not learn compositions of features. It learns that things are absent.

Greedy saturation is the deeper cause. E2 separates architecture from training. Given features that are actually good, the stack clears a ceiling a single layer provably cannot (99.4% against 75%, at a matched channel count). Given features produced by the greedy scheme, it does not (64.8%). The architecture is not what fails.

What survives

Two things are worth keeping from the proposal. First, stacking convolutional Tsetlin layers is genuinely more expressive than one layer — demonstrated, not argued, on a task whose one-layer ceiling is known. Second, *keeping the spatial map* rather than globally OR-pooling it is what makes relational concepts reachable at all: on *near* the spatial stack reaches 76.9% while the global-OR stack is pinned at chance whatever features it is given. The proposal's distinctive architectural choice is the right one. Its training scheme is not.

7.3 What would have to change

The greedy supervised scheme in the proposal couples two things that want opposite settings. Layer 1 is trained to classify, which rewards *specific* clauses — many included literals, rare firing — and layer 2 needs *general* clauses whose activations are common enough to be conjoined. E0 (§4) measures that coupling directly: across the sweep, every increase in layer-1 density costs layer-1 accuracy. A design that wants both cannot get them from one objective.

Three directions follow, in increasing order of how much they change the proposal.

- **Give layer 1 a different objective.** Nothing forces a feature layer to be trained by classification. A reconstruction objective — clauses that predict held-out features of a patch from the rest, as in the Tsetlin machine autoencoder — preserves information rather than discarding whatever did not help the local classifier, and it has no reason to drive clauses towards extreme specificity. This is the direct analogue of the unsupervised greedy pre-training that made layer-wise deep learning work in the first place.
- **Calibrate density explicitly.** Rather than hoping s lands somewhere usable, fix a target per-channel firing rate and adjust each clause towards it — a Boolean analogue of normalisation. The measurement in §4 is the diagnostic such a scheme would optimise.
- **Propagate credit instead of freezing.** The interesting question the proposal defers (its §23, Q9) is whether feedback can reach a lower layer at all. Clause inclusion is a usable credit path: if a layer-2 clause receives Type I feedback and includes the literal “channel k active at offset (dy, dx) ”, then layer-1 clause k can receive Type I feedback at that position. That is a discrete, rule-based analogue of backpropagation, it requires no gradients, and it removes the greedy-saturation problem by construction. It is a bigger change than anything tested here, and it is the part of the idea most worth pursuing.

7.4 Limitations of this evaluation

Scale. These are small Tsetlin machines by CIFAR-10 standards — hundreds of clauses where the literature uses thousands — chosen so that ten arms across three seeds fit in a few GPU-hours. Absolute accuracies are correspondingly modest and should not be read as CIFAR-10 results for Tsetlin machines. The comparison is between arms at a matched clause budget, and that is what the conclusions rest on.

One stack depth, and a gap between the two datasets. Only two-layer stacks were run. E2 shows CIFAR-10 was, in part, the wrong place to look: its classes are not obviously concepts that need a conjunction or a relation between separated parts, which is the only thing the second layer supplies. That makes the CIFAR-10 result a statement about the greedy training scheme on a natural-image task, not about the architecture — E2 is what speaks to the architecture. Whether a relational advantage of the kind *near* isolates ever shows up on natural images is untested here, and is the obvious next question.

The oracle is an upper bound, not a method. `build_oracle_l1` writes the right features in by hand. It establishes that the architecture can use good features, and therefore that training is the binding constraint; it does not suggest how to obtain such features without knowing the answer. The three directions above are proposals, not results.

Hyper-parameters. T was fixed at the clause count and not tuned per arm, and layer-2 specificity was tied to layer-1’s. A wider search could move the numbers; it is unlikely to close the gaps reported in §5, which are large relative to seed spread.

8 Reproducing this report

Everything lives in `experiments/mctm/`. The report reads only `results/*.json`; re-running the experiments and rebuilding regenerates every figure, table and inline number.

```
cd experiments/mctm

# E0 -- layer-1 density diagnostic (writes results/density_sweep.json)
python sweep_density.py --epochs 5 --subset 20000

# E1 -- the arms, three seeds each (writes results/<arm>_seed<k>.json)
./run_all.sh

# report: JSON -> data/*.csv + macros.tex -> PDF
cd report && make data && make
```

Files. `mctm.py` is the implementation (§2); `cifar.py` loads and caches the Booleanized dataset; `run.py` is the experiment driver, one arm per invocation; `sweep_density.py` is E0; `report_data.py` turns results into `report/data/*.csv` and `report/macros.tex`; `smoke.py` exercises every code path on synthetic data and is the fastest check that the machinery still works.

Environment. torchsetlin 0.1.3, PyTorch with CUDA, one NVIDIA RTX 3090. Layer 1 is cached per `(seed, config)` under `.ckpt/` because four arms share it; reported training time is the sum of recorded per-epoch times across both stages, so it is unaffected by that cache.

Upstreaming. The only library-level change the MCTM needs is a public accessor for the spatial map that `_ConvMixin._evaluate` already computes — a single method alongside `evaluate_clauses`. The prototype deliberately reaches into the private hooks instead, to keep an unproven idea off the public API surface.